AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

From a paper algorithm to Python

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

Before you start

Four packs contain six counters each. What steps find and show the total?

Plan time to read, investigate and explain. Keep predictions separate from observations.

STUDENT LESSON · 1 OF 18

Your mission

> Start with the target: Translate a short paper calculation into a Python script. Try the recall before opening notes.

Your learning target

I can translate a short paper calculation into a Python script.

I can predict and explain the order of executed statements.

I can keep expected and observed **output** separate when checking a **program**.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

> Four packs contain six counters each. What steps find and show the total?

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Translate a short paper calculation into a Python script

Predict and explain the order of executed statements

Keep expected and observed **output** separate when checking a **program**

Retrieve what you already know

> Four packs contain six counters each. What steps find and show the total?

My starting explanation

This week's end goal

Trace from a paper **algorithm** to python and explain the changed **program** before running it.

Notice the evidence

> Investigate the supplied From a paper algorithm to Python case. Separate observations, given facts and assumptions before choosing an explanation.

In Grade 6 you traced rules on paper. Now you will write a small Python **program** and check what it actually does. Begin with a counter-packing calculation, not the whole activity adviser.

The requirement is to find the total counters in equal packs and display that total. For this first **program**, the supplied quantities are whole, nonnegative numbers; it does not yet validate typed **input**.

Use a teacher-prepared Python 3 editor that can run a saved .py file. Open a new file called counters.py, type the code exactly as shown in the next section, save it, and use the editor's Run command.

Read the **output** area separately from the source editor. No account, AI tool, package installation or personal data is needed. If a device is unavailable, trace on paper and label the result predicted rather than executed.

> **List the given facts and one fact that is not supplied.**

STUDENT LESSON · 3 OF 18

See how it works

> Follow the mechanism for From a paper algorithm to Python. Connect each labelled input, deciding step and result.

A calculation from paper to output

PAPER STEP	PYTHON INSTRUCTION	EFFECT
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

```
packs = 4
per_pack = 6
total = packs * per_pack
print(total)
```

Read the script from top to bottom. The equals sign assigns the value on its right to the name on its left. The star means multiplication. print(total) displays the value of total; it does not display the letters in the name.

Type ordinary straight characters and keep the parentheses. The Python prompt symbols >>> are not part of this saved script.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

Compare the cases

> Compare the supplied cases for From a paper algorithm to Python. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual **output**, any error and what changed. If working on paper, label both results as traces.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Change one thing in the pictured case

> **Change one thing**
>
> Change: Change spare to 5 while keeping both tray **input**s fixed.

> **Predicted result**
>
> **Prediction**: The printed total becomes 29.

> **Why the result follows**
>
> Reason: 4*6 remains 24; adding five gives 29.

Words that unlock the idea

> Use the vocabulary of From a paper algorithm to Python to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
Algorithm	An ordered procedure for solving a stated problem
Program	Instructions written in a language a computer can execute
Interpreter	Software that executes Python instructions
Assignment	Binding a name to the value of an expression
Output	Information a program displays or returns

> **Use two terms to explain a step in this week's example.**

STUDENT LESSON · 6 OF 18

Follow a worked example

> Follow the worked trace for From a paper algorithm to Python, predict a step, then compare with its explanation.

After line	Names and values	Displayed output
packs = 4	packs is 4	Nothing yet

After line	Names and values	Displayed output
per_pack = 6	packs is 4; per_pack is 6	Nothing yet
total = packs * per_pack	total is 24	Nothing yet
print(total)	Values are unchanged	24

Worked explanation · From a paper algorithm to Python

Write 24 as the expected **output** before running. Then copy the actual **output** into a separate column. A matching result supports this case, not every possible **input**.

A saved file may contain an old edit: save again and rerun the whole script. Do not type the expected number into the **output** record without running.

Why is there no displayed output after the third line? What changes if print(total) is placed first in a fresh run, before any assignment? Explain using which names exist.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: From a paper algorithm to Python

Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

The facts we will use

Supplied facts

Python 3: trays=4; per_tray=6; spare=2; total=trays*per_tray+spare; print(total). All quantities count seeds.

AI ACADEMY / PYTHON-BRIDGE/1

From a paper algorithm to Python

Start with the supplied facts
Follow each arrow. Ask which fact allows the next step.

1. Name the inputs

Four trays hold six seeds each; two extra seeds are separate.

2. Translate the plan

Multiply tray count by seeds per tray, then add the spare count.

3. Trace the assignment

4*6+2 is 26; total stores 26 and print displays it.

CHANGE ONE THING
Change spare to 5 while keeping both tray inputs fixed.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Name the inputs	Four trays hold six seeds each; two extra seeds are separate.
2. Translate the plan	Multiply tray count by seeds per tray, then add the spare count.
3. Trace the assignment	4*6+2 is 26; total stores 26 and print displays it.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

What this example does not establish

Keep the stated limit

The calculation assumes identical tray sizes and measured counts; it cannot detect a miscounted tray.

Keep the stated limit

Read the labelled picture: Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

Keep the stated limit

Change one thing: Change spare to 5 while keeping both tray **input**s fixed. Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the From a paper algorithm to Python claim and explain a repair using the supplied evidence.

Check the claim against the facts

Change only packs to 5. Predict the result from five groups of six, save, then run the entire file again. The expected **output** is 30. Keep the original four-pack result beside the new one and label the changed **input**.

A comment can explain an instruction: Python ignores text after # outside a string. A comment does not change the calculation.

```
packs = 5  # five equal packs
per_pack = 6
total = packs * per_pack
print(total)
```

Check the claim against the facts

A **program** can run successfully and still solve the wrong problem. Replacing * with + would display 11 for this **input**, but addition does not count five groups of six.

Use the requirement and a paper calculation to check meaning; the absence of an error message is not proof of correctness.

Locate the earliest unsupported step

Move print(total) to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

State one repair and the checks you would rerun.

STUDENT LESSON · 8 OF 18

Practise together

Practise From a paper algorithm to Python with support, then explain the deciding step in your own words.

Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line displays information?

Why is there no displayed output after the third line? What changes if print(total) is placed first in a fresh run, before any assignment? Explain using which names exist.

Explain your prediction to a partner before comparing results.

Practise with less help: Predict then observe

Use workbook W2 for the connected example “From a paper **algorithm** to Python”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for From a paper **algorithm** to Python. Proposed explanation: “An equals sign displays the answer”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

> Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

STUDENT LESSON · 9 OF 18

Run the investigation

> Investigate From a paper algorithm to Python. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual **output**, any error and what changed. If working on paper, label both results as traces.

```
packs = 4
per_pack = 6
total = packs * per_pack
print(total)
```

```
packs = 5  # five equal packs
per_pack = 6
total = packs * per_pack
print(total)
```

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in From a paper algorithm to Python. Explain why each step is justified.

Explain → apply → challenge

> Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line displays information?

Move print(total) to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

Explain the difference between a successful run and a correct calculation. State one limit of this **program** and what you would keep so another person can check your work.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Python readiness portfolio. Keep an annotated decision and the evidence supporting it.

Add a labelled from a paper **algorithm** to python **evidence** sheet to Python readiness portfolio. Show how you can translate a short paper calculation into a python script.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

Save the file with the **input** values, expected and observed **output**s, and a short explanation. This **program** follows a written multiplication rule.

It has not trained a model, predicted somebody's preferences or checked that real packs contain the stated number. Its conclusion depends on the supplied data. Week 2 examines the values and types behind these names.

A script runs without an error and displays 11 for five packs of six. Is it correct for the stated requirement? Give the required result, the likely wrong operation and one check after repair. State whether your evidence came from a device run or a paper trace.

Explain the difference between a successful run and a correct calculation. State one limit of this **program** and what you would keep so another person can check your work.

My limited claim and a useful next check

STUDENT LESSON · 13 OF 18

Study a complete case

> Revisit the complete From a paper algorithm to Python case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

A calculation from paper to output

PAPER STEP	PYTHON INSTRUCTION	EFFECT
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

Try a new case independently

> Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Trace variables and a change

Predict the printed value. Change only packs to 5 and predict again. Explain which statements apply a **written rule**.

```
packs = 3
per_pack = 7
total = packs * per_pack
print(total)
```

> **My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

A fictional shelf has 3 trays of 7 tiles each. Write a four-line Python script using trays, per_tray and tiles. Predict its **output**. Then change only trays to 0 and predict again. Explain why adding the two quantities is wrong.

> **My independent procedure, result and limitation**

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

STUDENT LESSON · 15 OF 18

Your lesson route

> Use this map to navigate From a paper algorithm to Python, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

By the end, I can	How I will show it
Translate a short paper calculation into a Python script	Explain the deciding step, show a trace or test, and state one limit.
Predict and explain the order of executed statements	Explain the deciding step, show a trace or test, and state one limit.
Keep expected and observed output separate when checking a program	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain From a paper algorithm to Python without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Translate a short paper calculation into a Python script

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? 4*6+2 is 26; total stores 26 and print displays it.

1. Trace the **assignment**

2. Name the **input**s

3. Translate the plan

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Trace the **assignment** 4*6+2 is 26; total stores 26 and print displays it. Python follows the arithmetic instructions.

This translation automates a declared counting rule; it does not train a model from examples.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Change spare to 5 while keeping both tray **input**s fixed. Which conclusion is supported by the supplied facts?

1. The printed total becomes 29.

2. We can decide the result without examining the changed **input** or the procedure.

3. This one example guarantees correct results for every future **input**.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: The printed total becomes 29. 4*6 remains 24; adding five gives 29.

Repair a missing building block

Review Debugging Strategies (ai-2/6). Goal: Design revealing tests for a function. Short practice: Which test most directly isolates the handling of loose books, and what should it produce?.

Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

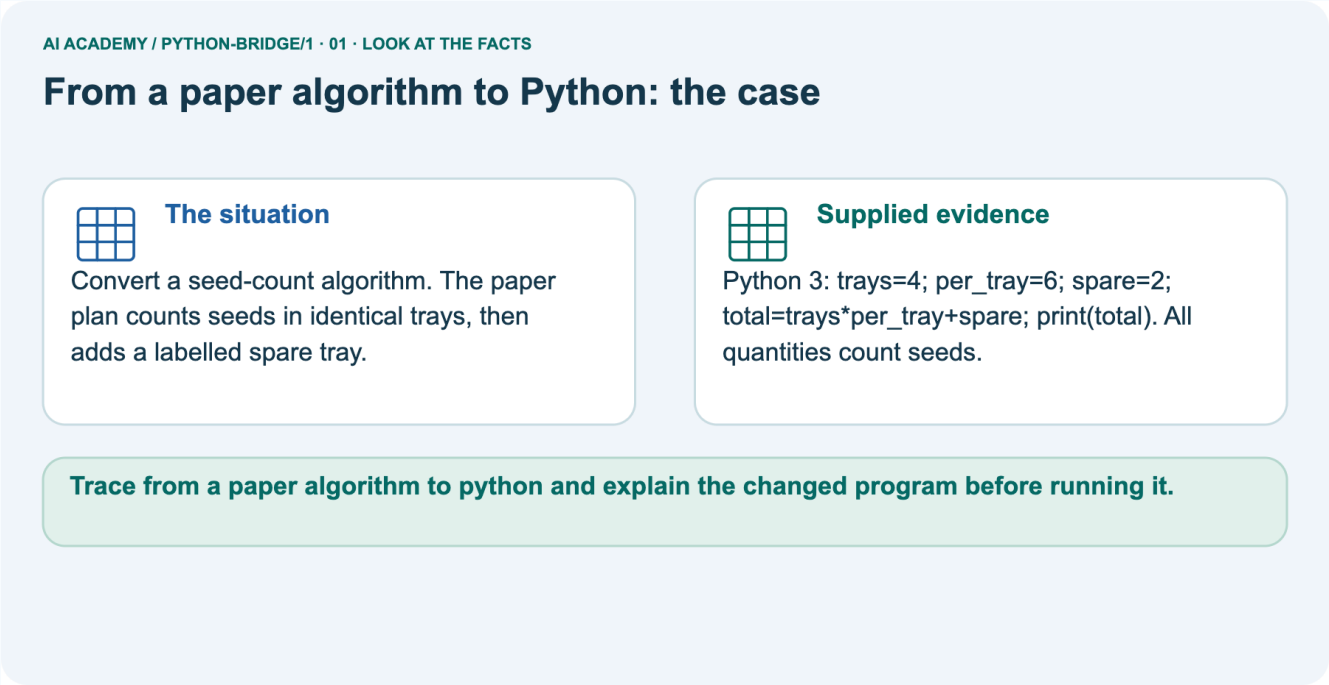
Drafts stay in this browser when storage is available; clear drafts on a shared device.

STUDENT LESSON · 18 OF 18

Visual concept journey

> Explain each From a paper algorithm to Python graphic using its labels, deciding evidence and relevant limitation.

01. From a paper algorithm to Python: the case



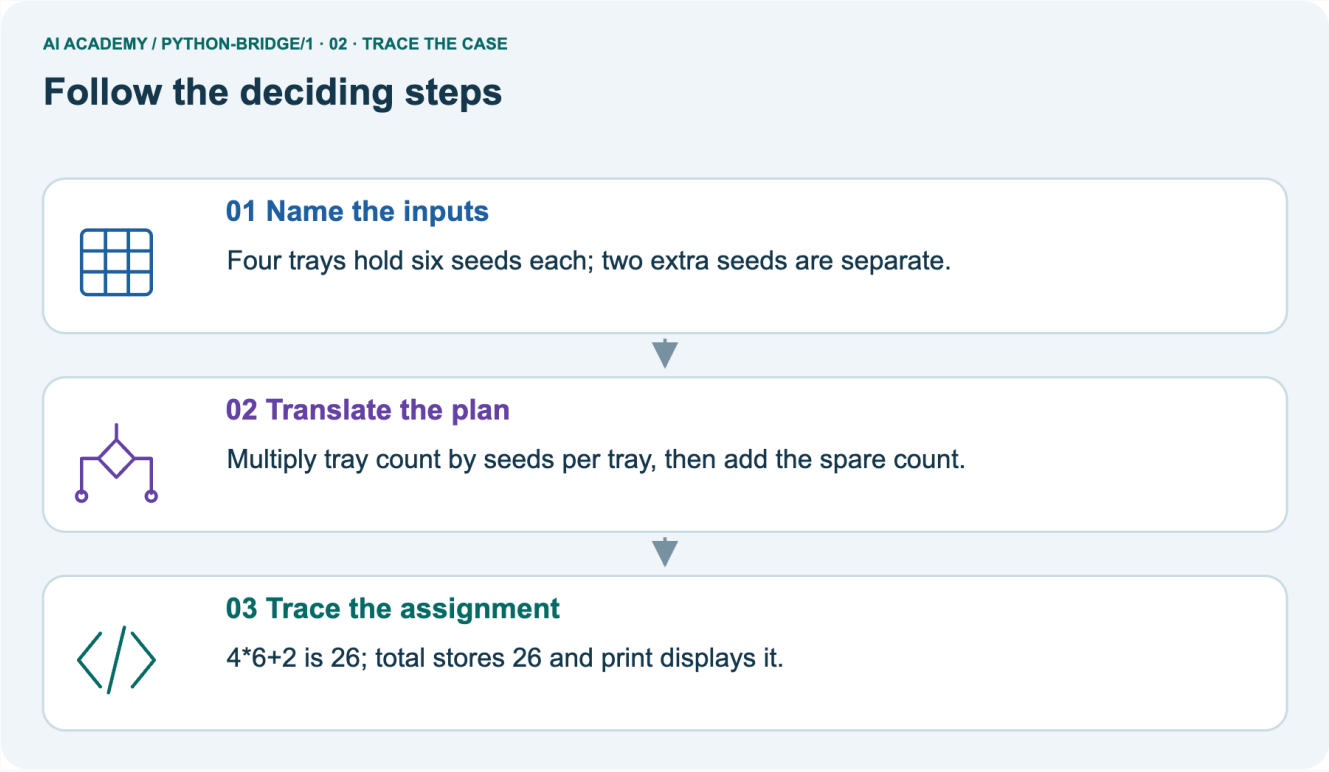
Follow the labelled picture with the matching explanation. Open the vector diagram

Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray. Supplied facts: Python 3: trays=4; per_tray=6; spare=2; total=trays*per_tray+spare; print(total).

All quantities count seeds.

> Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Name the **input**s: Four trays hold six seeds each; two extra seeds are separate. → Translate the plan: Multiply tray count by seeds per tray, then add the spare count. → Trace the **assignment**: 4*6+2 is 26; total stores 26 and print displays it.

> Point to each numbered stage and explain which recorded **input** it uses.

03. A calculation from paper to output

AI ACADEMY / PYTHON-BRIDGE/1 · 03 · INSPECT THE LESSON MODEL

A calculation from paper to output

Paper step	Python instruction	Effect
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

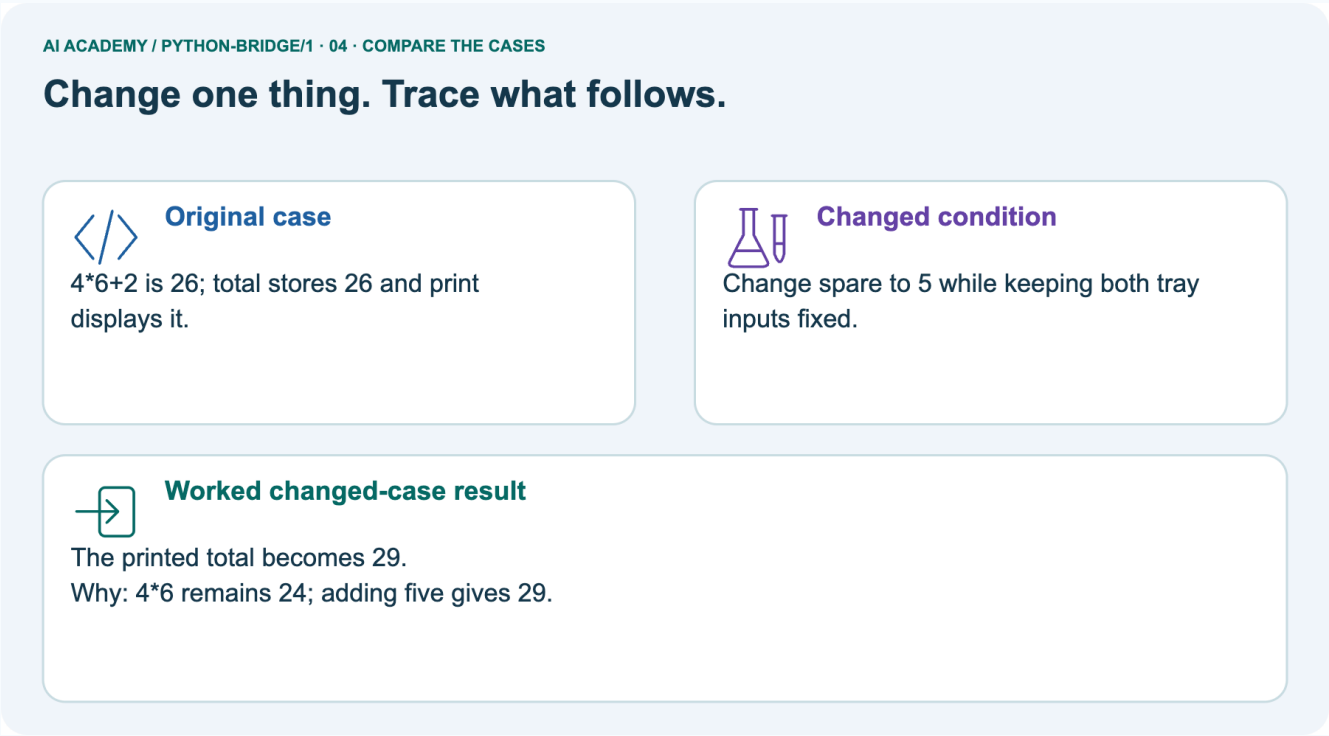
Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Follow the labelled picture with the matching explanation. Open the vector diagram

A calculation from paper to **output**. Record pack count | packs = 4 | Name holds 4. Multiply two quantities | total = packs * per_pack | Name holds product. Show the answer | print(total) | Displays the value

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



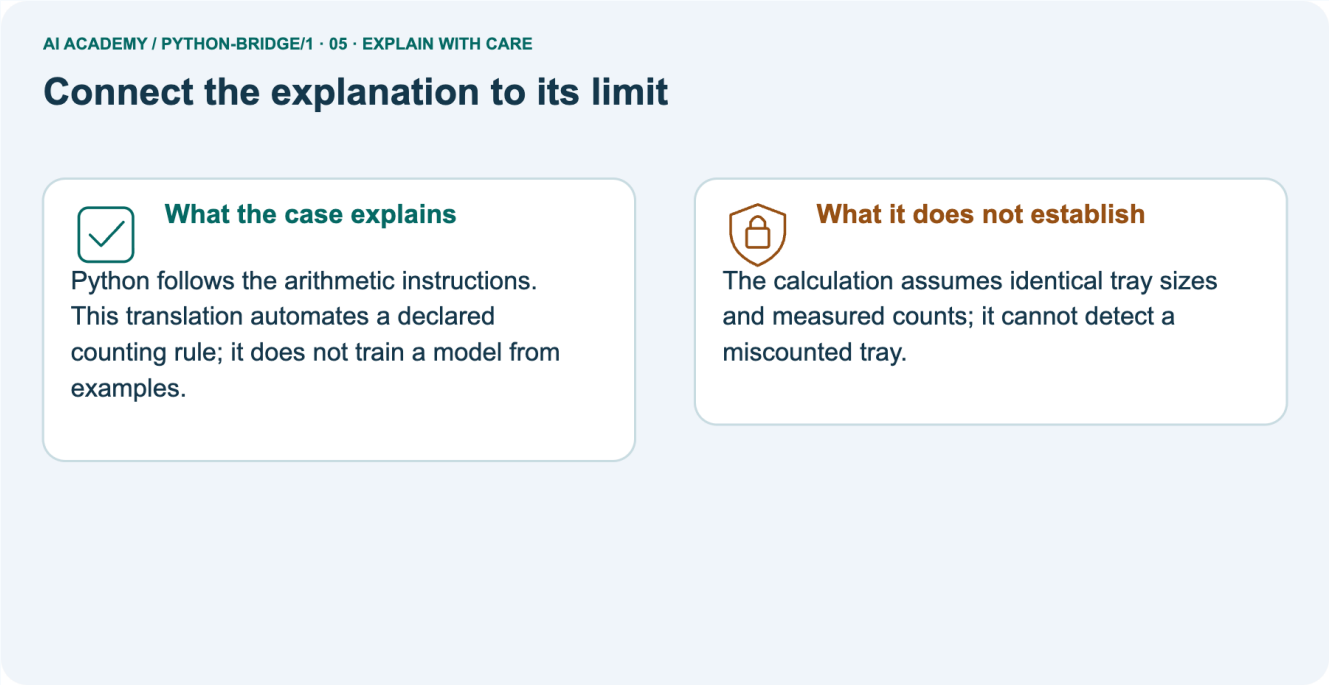
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: 4*6+2 is 26; total stores 26 and print displays it. Changed condition: Change spare to 5 while keeping both tray **input**s fixed. Expected result: The printed total becomes 29.

Reason: 4*6 remains 24; adding five gives 29.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



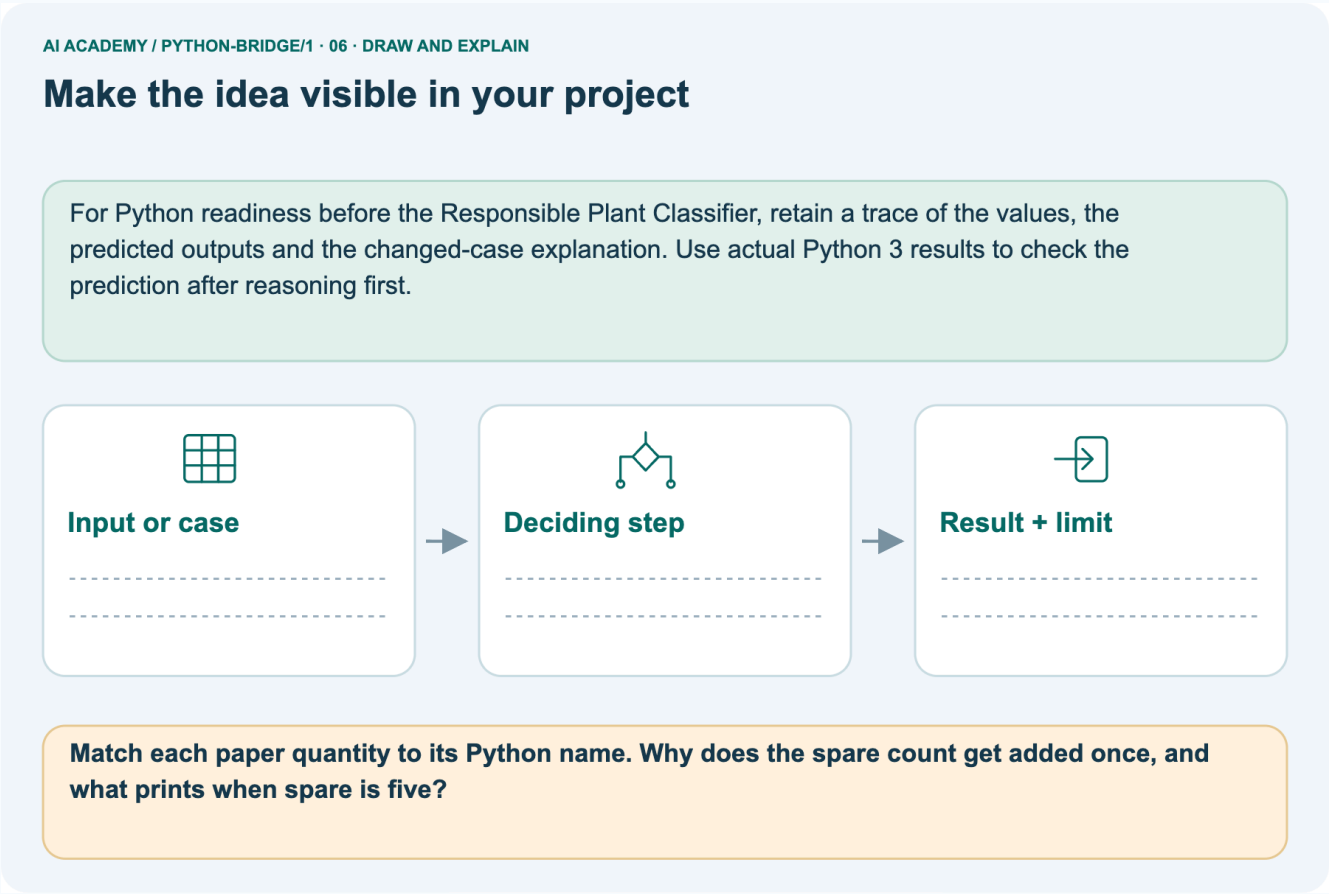
Follow the labelled picture with the matching explanation. Open the vector diagram

Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

Limit: The calculation assumes identical tray sizes and measured counts; it cannot detect a miscounted tray.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.